SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

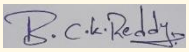
Department of Computer Science and Engineering

CO2 ASSESSMENT TOOL 1 – Scenario-Based Requirement Analysis

Course Code:	CSA10 / CSA1063	**Course Title:**	Software Engineering
CO Assessed:	CO2	**Assessment:**	Assessment Tool 2 – Scenario-Based Requirement Analysis
Weightage:	20%	**Total Marks:**	25
CO2 Statement:	*Perform detailed requirements analysis, design scalable architectures, and prioritize features with a focus on cross-disciplinary skills*		
Student Name:	B. Charan Kumar Reddy	**Reg. No.:**	192472195
Date:	22/10/2026	**Duration:**	60 minutes

Code of Conduct

I B. Charan Kumar Reddy (192472195) *certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the Student: 

Annexure A – Scenario-Based Requirement Analysis

Instructions to Students:
Each student will be assigned an individual software project problem statement. Analyze the given scenario and prepare a complete Software Requirement Analysis document by following the steps below.

Question

Based on the **assigned problem statement**, perform a **Scenario-Based Requirement Analysis** and prepare a Software Requirement Specification (SRS) document. Your analysis should include the following:

1. **Study and Analyze the Scenario**
   - Carefully understand the given problem statement.
   - Identify the business domain, stakeholders, existing challenges, and system objectives.
2. **Identify Functional and Non-Functional Requirements**
   - List all functional requirements required by the proposed system.
   - Identify suitable non-functional requirements such as performance, security, reliability, usability, scalability, maintainability, and availability.
3. **Identify Actors and Use Cases**
   - Identify all primary and secondary actors interacting with the system.
   - List the major use cases associated with each actor.
   - Draw a Use Case Diagram representing the interactions between actors and the system.
4. **Prepare the Software Requirement Specification (SRS)**
   Develop an SRS document containing:
   - Introduction
   - Problem Description

- Scope of the System
- Functional Requirements
- Non-Functional Requirements
- Use Case Descriptions
- Data Requirements
- External Interface Requirements
- System Features

5. **Identify Assumptions and Constraints**
   - List the assumptions considered while developing the system.
   - Identify technical, operational, economic, legal, and time-related constraints affecting the project.
6. **Validate Requirement Completeness and Consistency**
   - Verify whether all stakeholder requirements have been addressed.
   - Check for ambiguity, redundancy, inconsistency, and missing requirements.
   - Suggest improvements to enhance the quality and completeness of the requirement specification.

Answer

Scenario-Based Requirement Analysis

Problem Statement: Autonomous Coding Agent Security and Software Supply-Chain Protection Platform

The proposed platform monitors AI coding agents, third-party dependencies, repositories, and CI/CD workflows to detect suspicious activities such as unauthorized actions, secret exposure, malicious dependency changes, and abnormal repository modifications. It supports risk assessment, human approval, security auditing, and isolation or rollback of suspicious changes.

1. Study and Analyze the Scenario

a) Business Domain

The project belongs to the domains of software engineering, cybersecurity, artificial intelligence, and software supply-chain security. Modern software development uses AI coding agents, open-source dependencies, source-code repositories, and automated CI/CD pipelines. These technologies improve productivity but introduce security risks when agents are misused or software components are compromised.

b) Stakeholders

The major stakeholders involved in the system are:

- Developers: Create and modify code using AI coding agents.
- Security Administrators: Define security policies, monitor alerts, and investigate incidents.
- DevOps Engineers: Manage CI/CD pipelines, builds, and deployments.
- Project Managers: Manage project progress, resources, and security priorities.
- AI Coding Agents: Generate code, modify files, install dependencies, and execute authorized commands.
- System Administrators: Manage system permissions, infrastructure, and recovery.
- Organization Management: Define security requirements and evaluate risks.

c) Existing Challenges

The major challenges in the current development environment include:

1. Unauthorized AI-agent actions and tool execution.
2. Vulnerable or compromised third-party dependencies.
3. Accidental exposure of API keys, passwords, and access tokens.
4. Privilege escalation and unauthorized access.
5. Malicious build activities and CI/CD pipeline manipulation.
6. Abnormal repository modifications.
7. Fragmented security monitoring and delayed incident response.

d) System Objectives

The main objectives of the proposed system are:

- Continuously monitor AI agents, repositories, dependencies, and CI/CD workflows.
- Detect suspicious activities and calculate risk scores.
- Maintain Software Bills of Materials (SBOMs) and security audit trails.
- Enforce dependency policies and require approval for sensitive actions.
- Isolate suspicious changes and support rollback.
- Improve software security, reliability, and accountability.

2. Functional and Non-Functional Requirements

a) Functional Requirements

Functional requirements describe the operations that the system must perform.

ID	Functional Requirement
FR01	The system shall monitor AI-agent actions and tool executions.
FR02	The system shall track repository modifications and identify abnormal changes.
FR03	The system shall scan third-party dependencies for known vulnerabilities and policy violations.
FR04	The system shall generate and maintain Software Bills of Materials (SBOMs).
FR05	The system shall detect exposed secrets such as API keys and credentials.
FR06	The system shall detect unauthorized actions and suspicious privilege changes.
FR07	The system shall monitor CI/CD builds and pipeline configuration changes.
FR08	The system shall calculate risk scores and generate security alerts.
FR09	The system shall request human approval for sensitive actions.
FR10	The system shall block or isolate suspicious activities according to security policies.
FR11	The system shall support rollback of suspicious repository changes.
FR12	The system shall maintain audit logs and generate security reports.

b) Non-Functional Requirements

Non-functional requirements describe the quality attributes and constraints of the system.

Requirement	Description
Performance	Security checks should produce results promptly without significantly delaying development.
Security	The platform shall protect source code, credentials, policies, and audit records.
Reliability	Monitoring and incident-response functions should operate consistently.
Usability	Alerts, risk scores, and approval requests should be easy to understand.

Scalability	The system should support increasing numbers of repositories, agents, and dependencies.
Maintainability	Components should be easy to update and test independently.
Availability	The platform should remain available during normal development and CI/CD operations.

3. Actors and Use Cases

a) Actors

The primary actors are Developer, Security Administrator, DevOps Engineer, and System Administrator.

The supporting systems are AI Coding Agent, Source-Code Repository, Dependency Registry, and CI/CD Pipeline.

b) Major Use Cases

The major use cases include:

- Monitoring AI-agent actions.
- Monitoring repository changes.
- Scanning dependencies.
- Generating and updating SBOMs.
- Detecting secrets and unauthorized actions.
- Monitoring CI/CD builds.
- Calculating risk scores and generating alerts.
- Approving or rejecting sensitive actions.
- Isolating suspicious changes.
- Rolling back unsafe changes.
- Viewing audit logs and security reports.

c) Use Case Diagram

The use case diagram will be inserted here separately.

The diagram should show the external actors, the system boundary, and the major use cases of the Autonomous Coding Agent Security and Software Supply-Chain Protection Platform.

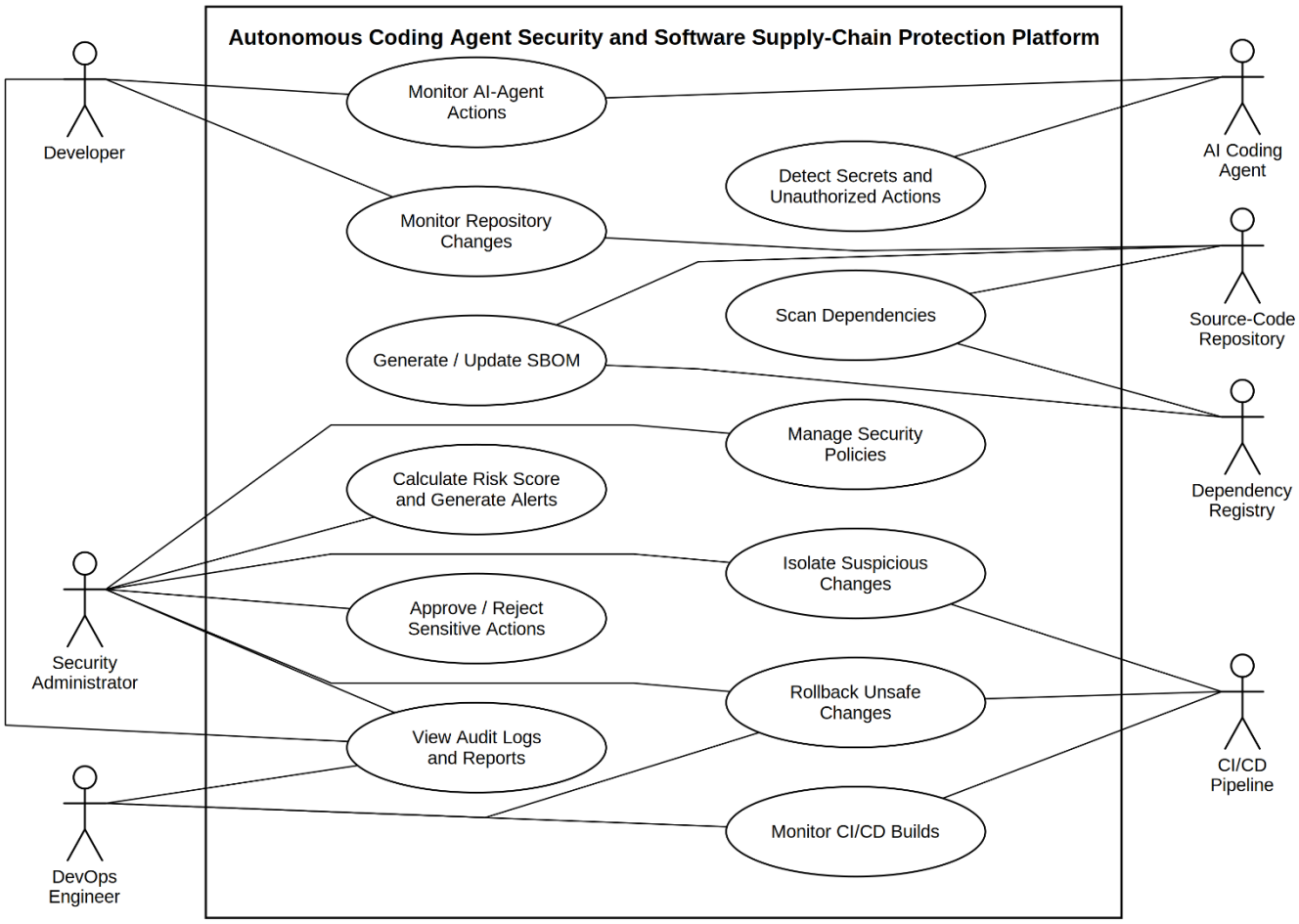


Figure 1: UML Use Case Diagram for the Autonomous Coding Agent Security and Software Supply-Chain Protection Platform.

4. Software Requirement Specification (SRS)

a) Introduction

The purpose of the system is to protect AI-assisted software development by monitoring agent activities, dependencies, repositories, and CI/CD workflows. This Software Requirement Specification defines the system's purpose, scope, functional requirements, quality requirements, interfaces, and expected behavior.

b) Problem Description

AI coding agents can automatically perform development operations, while software projects depend on third-party packages and automated pipelines. Unauthorized agent actions, compromised dependencies, secret exposure, and malicious build activities can introduce security threats. The proposed platform provides integrated monitoring, risk assessment, human approval, and incident-response capabilities to reduce these risks.

c) Scope of the System

The system covers AI-agent monitoring, repository monitoring, dependency analysis, SBOM management, CI/CD security, risk scoring, human approval, incident response, rollback, and audit reporting. It complements existing development tools rather than replacing them.

d) System Requirements

The system shall:

1. Record AI-agent actions and tool executions.
2. Detect abnormal repository modifications.
3. Scan dependencies and enforce security policies.
4. Generate and update SBOMs.
5. Detect exposed secrets and unauthorized access.
6. Monitor CI/CD pipeline activities.
7. Calculate risk scores and issue alerts.
8. Request human approval for sensitive actions.
9. Isolate suspicious activities and support rollback.
10. Store audit records and generate reports.

e) Quality Requirements

- Performance: Minimize delays during monitoring and security checks.
- Security: Protect credentials, source code, and security records.
- Reliability: Ensure consistent monitoring and dependable incident handling.
- Usability: Provide clear alerts and understandable risk classifications.
- Scalability: Support multiple projects and growing dependency inventories.
- Maintainability: Enable independent updates to system components.
- Availability: Provide access during expected development operations.

f) Use Case Descriptions

Use Case 1: Scan Dependencies

- Actor: Security Administrator.
- Precondition: A project and its dependency manifest are available.
- Main Flow: The system reads dependencies, checks available vulnerability information and configured policies, and produces findings.
- Postcondition: Scan results and detected risks are recorded.

Use Case 2: Approve Sensitive Agent Action

- Actor: Security Administrator.
- Precondition: An AI agent requests an action requiring approval.
- Main Flow: The system evaluates the request, presents its details, and waits for an authorized decision. The action proceeds only if approval is granted.
- Postcondition: The approval decision is recorded in the audit trail.

Use Case 3: Isolate Suspicious Change

- Actor: Security Administrator or automated incident-response process.
- Precondition: A repository or development activity is identified as high risk.
- Main Flow: The system generates an alert, applies the configured blocking or isolation action, and records the incident.
- Postcondition: The suspicious activity is contained and made available for investigation.

g) Data Requirements

The system shall maintain the following data:

- User and role information.
- AI-agent action logs.
- Repository and change records.
- Dependency inventories and SBOMs.
- Vulnerability and secret-detection findings.
- CI/CD build and pipeline events.
- Risk scores and security alerts.
- Approval decisions.
- Incident and rollback records.
- Audit logs and security reports.

h) External Interface Requirements

- User Interface: A dashboard for alerts, risk scores, approvals, and reports.
- Repository Interface: Integration with version-control systems to retrieve change events.
- Dependency Interface: Integration with package registries and vulnerability data sources.
- CI/CD Interface: Integration with build pipelines to receive events and enforce security checks.
- AI-Agent Interface: Event reporting and controlled permission or approval requests.

i) System Features

The major features include continuous monitoring, dependency scanning, SBOM management, secret detection, risk scoring, human approval, CI/CD security, threat isolation, rollback, and audit reporting.

5. Assumptions and Constraints

a) Assumptions

- Users have valid accounts and assigned permissions.
- The platform can access the necessary repository and CI/CD events.
- Dependency vulnerability information is available from a suitable source.
- AI coding agents can provide relevant action information or be monitored through supported integrations.
- Authorized personnel are available to review sensitive approval requests.
- Repositories retain sufficient history to support investigation and recovery.

b) Constraints

Type	Constraint
Technical	Different repositories, AI agents, package managers, and CI/CD tools may require separate integrations.
Operational	Monitoring and approval should not unnecessarily interrupt development.
Economic	Implementation, hosting, and security-data processing are limited by available resources.
Legal	Source code, credentials, and audit data must be handled according to applicable privacy and security obligations.
Time	The project must be completed within the assigned development and assessment schedule.

6. Validate Requirement Completeness and Consistency

a) Stakeholder Coverage

The requirements cover developers, security administrators, DevOps engineers, system administrators, AI coding agents, repositories, dependency registries, and CI/CD pipelines. Their responsibilities and interactions are represented in the functional requirements and use cases.

b) Ambiguity, Redundancy, and Inconsistency Checks

- Each functional requirement has a unique identifier.
- Monitoring, detection, approval, and incident response are defined as distinct but connected functions.
- Risk levels should have documented thresholds and corresponding actions.
- Sensitive actions must not execute before required approval.
- Rollback should apply only to changes that can be safely identified and restored.
- Security alerts should contain sufficient event details to support investigation.

c) Improvements to Requirement Quality

- Define measurable performance targets based on expected system load.
- Specify supported AI agents, repositories, package registries, and CI/CD platforms.
- Define risk-scoring rules and thresholds.
- Define approval permissions, timeouts, and rejection behavior.
- Test secret detection and dependency scanning using representative test cases.
- Define rollback conditions and recovery procedures.
- Review requirements with stakeholders and maintain traceability from each requirement to its test case.

Conclusion

The scenario-based requirement analysis identifies the problem domain, stakeholders, functional and non-functional requirements, actors, use cases, and SRS for the Autonomous Coding Agent Security and Software Supply-Chain Protection Platform. The assumptions, constraints, and validation checks help ensure that the requirements are clear, consistent, testable, and aligned with stakeholder needs. This provides a foundation for designing and implementing the proposed security platform.

Rubrics for Calculation

Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
Scenario Understanding and Problem Analysis (30%)	Demonstrates thorough understanding of the scenario, stakeholders, objectives, and business context with clear analysis.	Demonstrates good understanding with minor omissions.	Basic understanding with limited analysis.	Poor understanding; major aspects missing or incorrect.
Functional and Non-Functional Requirement Identification (25%)	Clearly identifies comprehensive, accurate, and well-classified functional and non-functional requirements.	Identifies most requirements with minor classification errors.	Identifies only basic requirements; several omissions.	Requirements are incomplete, inaccurate, or poorly classified.
Actor Identification and Use Case Modelling (20%)	Correctly identifies all relevant actors and use cases; use case diagram is complete, accurate, and follows UML standards.	Most actors and use cases identified; diagram has minor errors.	Limited actors/use cases identified; diagram partially correct.	Actors/use cases missing or incorrect; diagram absent or inaccurate.
Software Requirement Specification (SRS) Documentation (15%)	SRS is well-structured, complete, professionally organized, and includes all required sections.	SRS is organized with minor missing sections or formatting issues.	SRS includes only essential sections with limited detail.	SRS is incomplete, poorly organized, or lacks essential components.
Assumptions, Constraints, and Requirement Validation (10%)	Clearly identifies realistic assumptions and constraints; validates requirements for completeness, consistency, and ambiguity with appropriate justification.	Identifies most assumptions and constraints; validation is adequate.	Limited assumptions/constraints; validation is superficial.	Assumptions, constraints, and validation are missing or incorrect.

Criteria	Mark
Scenario Understanding and Problem Analysis (30%)	/5
Functional and Non-Functional Requirement Identification (25%)	/5
Actor Identification and Use Case Modelling (20%)	/5

Software Requirement Specification (SRS) Documentation (15%)	/5
Assumptions, Constraints, and Requirement Validation (10%)	/5
TOTAL	/25